1. Model notation

A classifier is given an input x and predicts a label y. In sentiment classification, x might be the text of a product review and y its sentiment label: 1 for positive, 0 for negative.

The symbol theta denotes the model's learned parameters, for example the weights and bias of a logistic regression. Training adjusts theta; the inputs and labels are data.

We write p(y | x, theta) for the conditional probability the model assigns to label y given input x and parameters theta. The vertical bar reads 'given'.

In binary classification we abbreviate p = p(y = 1 | x, theta). The probability of class 0 is then 1 - p.

A probability is not a class label. To predict a class we apply a decision rule, commonly a threshold: predict 1 if p >= 0.5, otherwise predict 0. The probability describes the model's confidence; the threshold turns it into a decision.

Worked example: a model outputs p = 0.7 for the review 'Great battery life'. The model assigns probability 0.7 to the positive label and 0.3 to the negative label. With a 0.5 threshold it predicts label 1.

2. Dataset likelihood

Suppose we have n labelled examples (x_1, y_1), ..., (x_n, y_n). The likelihood of the parameters is the probability the model assigns to the observed labels.

If we assume the labels are conditionally independent given their inputs and theta, the joint probability factorises into a product: L(theta) = product over i of p(y_i | x_i, theta).

During fitting the observed labels are fixed. Theta is what varies, and changing theta changes each factor p(y_i | x_i, theta).

Conditional independence is a modelling assumption that makes the product valid. It is not a property guaranteed for every dataset; for example, near-duplicate reviews from the same author may be correlated.

Worked example: three examples receive true-label probabilities 0.9, 0.6 and 0.5. Under the independence assumption L(theta) = 0.9 x 0.6 x 0.5 = 0.27. A different theta would give different probabilities and a different likelihood.

3. Log objective

Products of many probabilities become tiny, so we work with the log likelihood: log L(theta) = sum over i of log p(y_i | x_i, theta). The log turns the product into a sum.

The logarithm is strictly increasing on positive values: if a > b > 0 then log a > log b. Therefore L and log L have the same maximisers.

Optimisation libraries usually minimise, so we negate: the negative log likelihood NLL(theta) = - log L(theta). Multiplying by -1 turns maximisation into minimisation, so the maximiser of L is the minimiser of NLL.

Using the mean NLL instead of the sum divides by the dataset size n, which is fixed and positive. This rescales the objective but preserves the minimiser.

Worked example: with true-label probabilities 0.9 and 0.5, log L = ln 0.9 + ln 0.5 = -0.105 - 0.693 = -0.798, so summed NLL = 0.798 and mean NLL = 0.399.

4. Meaning of binary cross-entropy

Binary cross-entropy (BCE) is the per-example negative log likelihood for binary labels. It penalises a model for assigning low probability to the true label.

Because -log q grows without bound as q approaches 0, a high-confidence wrong prediction receives a large loss. Predicting p = 0.01 for an example whose true label is 1 costs about 4.6, while p = 0.6 costs about 0.51.

BCE evaluates probabilities rather than merely counting correct labels. Two models with the same accuracy can have different BCE because they assign different probabilities to the true labels.

A low BCE is not a certificate that the predicted probabilities are calibrated; calibration needs a separate check.

Worked example: for true label 1, model A predicts 0.95 and model B predicts 0.55. Both are correct at a 0.5 threshold, but A's loss is -ln 0.95 = 0.051 and B's is -ln 0.55 = 0.598.

5. Calculation and application

BCE(y, p) = -[y log p + (1 - y) log(1 - p)], using natural logarithms and p in (0, 1).

If y = 1 the second term vanishes and BCE = -log p. If y = 0 the first term vanishes and BCE = -log(1 - p). The loss is never negative.

Remember that p is always the class-1 probability, not automatically the probability of the true label. For a label-0 example the true-label probability is 1 - p.

Worked example 1: for y = 1 and p = 0.8, BCE = -ln 0.8 = 0.22314 (about 0.223).

Worked example 2: for y = 0 and p = 0.4, BCE = -ln(1 - 0.4) = -ln 0.6 = 0.51083 (about 0.511).

For a dataset, the mean BCE averages the per-example losses. To compare two models on the same labelled examples, compute each model's mean BCE; the lower value indicates higher probability on the true labels.

Worked example 3: labels [1, 0] with class-1 probabilities [0.9, 0.2]. The losses are -ln 0.9 = 0.10536 and -ln(1 - 0.2) = -ln 0.8 = 0.22314, so the mean BCE is (0.10536 + 0.22314) / 2 = 0.16425 (about 0.164).

Optional extension: at p = 0 or p = 1 the log is undefined or the loss is infinite in the limit, so implementations clip probabilities or work with logits for numerical care.

6. Derivation

For a Bernoulli label, the model probability of the observed label can be written in one expression: p(y | x, theta) = p^y (1 - p)^(1 - y). For y = 1 this is p; for y = 0 it is 1 - p.

Take the negative log: -log[p^y (1 - p)^(1 - y)]. The log of a product is a sum of logs, and the log of a power brings the exponent down: -[y log p + (1 - y) log(1 - p)]. This is the binary cross-entropy.

Substituting into the dataset likelihood and taking the negative log gives the summed BCE over all examples.

Two-class one-hot cross-entropy is -sum over k of t_k log p_k. With targets t = (1 - y, y) and class probabilities (1 - p, p), the two terms are (1 - y) log(1 - p) and y log p, so the sum reduces to BCE.

Worked example: for y = 0 the derivation gives -[0 x log p + 1 x log(1 - p)] = -log(1 - p), matching the class-0 branch on page 5.